Université — Master 1 CHPS

Semestre 2 — 2025-2026

Yaya Toure

Master 1 CHPS — Parcours Calcul Haute Performance

Année universitaire 2025-2026

Résumé : Ce rapport présente les travaux réalisés lors du TD4 portant sur le profiling et la vectorisation de la fonction `ssgemm` (produit matriciel simple précision). En utilisant l'outil **Linux perf** (`perf stat` et `perf record`), nous avons analysé les compteurs hardware (cycles, instructions, cache-misses, dTLB-misses) de cinq versions fournies (v001 à v009). Puis, en répondant aux questions Q-4.1 et Q-4.2, nous avons conçu **mcblas010** : une version avec blocking modifié permettant la **vectorisation automatique AVX 256 bits** (8 floats simultanés), atteignant un gain de $\times$**19.9** par rapport à la référence naïve.

Contents

1 Contexte et architecture cible **3**
1.1 Architecture Intel Core i7 (Latitude 7480) . . . 3
1.2 Versions actives dans libmcblas004 . . . 3

2 Exercice 2 : Compilation (make RALL) **4**
2.1 Procédure de compilation . . . 4

3 Exercice 3 : Utilisation des outils de profiling **5**
3.1 Q-3.1.1 : Compteurs avec perf stat . . . 5
3.1.1 Modification du CMakeLists.txt . . . 5
3.1.2 Résultats comparatifs perf stat (N=1000) . . . 5
3.1.3 Tableau comparatif des compteurs hardware . . . 6
3.1.4 Analyse des compteurs . . . 6
3.1.5 Visualisation — Comparaison des compteurs hardware . . . 6
3.2 Q-3.1.2 : Profiling avec perf record . . . 6
3.2.1 Modification du CMakeLists.txt . . . 6
3.2.2 Résultats perf report (v009-O2, N=1000) . . . 6
3.3 Q-3.1.3 : Visualisation . . . 7

4 Exercice 4 : Vectorisation **8**
4.1 Q-4.1 : Modification du blocking pour la vectorisation . . . 8
4.1.1 Problème de vectorisation dans v007/v008/v009 . . . 8
4.1.2 Solution : mcblas010 — permuter pour stride-1 sur la boucle interne . . . 8
4.1.3 Confirmation de la vectorisation automatique . . . 8
4.2 Q-4.2 : Variation des paramètres . . . 9
4.2.1 Impact de la taille de bloc T (v010-O3, N=1000) . . . 9
4.2.2 Visualisation — Impact de la taille de bloc . . . 9
4.3 Comparaison finale des performances (N=1000) . . . 9
4.3.1 Visualisation — Comparaison toutes versions (échelle log) . . . 10

5 Conclusion Générale **11**
5.1 Récapitulatif des exercices . . . 11
5.2 Enseignements principaux . . . 11
5.3 Gains cumulés depuis TD3 . . . 11

1. Contexte et architecture cible

Ce TD fait suite au TD3 (permutations de boucles et loop tiling sur `ssgemm`). Il s'appuie sur le projet *libmcblas004* contenant 9 versions de la multiplication matricielle simple précision, ainsi que sur l'article de référence *"Compiler Transformations for High-Performance Computing"* (Bacon, Graham & Sharp, ACM Computing Surveys, 1994).

1.1. Architecture Intel Core i7 (Latitude 7480)

Composant	Valeur	Impact sur ssgemm
Cache L1d	64 KiB	Contrainte tiling : $3T^2 \times 4 \leq 65536$
Cache L2	512 KiB	2ème niveau de réutilisation
Cache L3	3 MiB	Débordement à N≥900
Registres YMM	256 bits	AVX : 8 floats simultanés
Compilateur	GCC 11.2.0	-O2/-O3, -march=native
Stockage	Row-major C	Stride-1 sur 2ème indice

1.2. Versions actives dans libmcblas004

Version	Signature	Technique	Particularité
v001	`float**`	Référence r→c→d	Ordre naïf
v002	`float**`	Permutation r→d→c	Stride-1 optimal
v007	`float* __restrict__`	Tiling + unroll 2r×4c	Td=Tr=Tc=32
v008	`float* __restrict__`	Tiling + vars locales Ar/Bd	Td=512
v009	`float* __restrict__`	Tiling + buffer Bbuffer	Td=1024
v010	`float* __restrict__`	**Tiling + AVX (Q-4.1)**	**Notre contribution**

2. Exercice 2 : Compilation (make RALL)

2.1. Procédure de compilation

Conformément au *README.org* fourni avec l'archive :

Listing 1: Compilation et exécution des benchmarks

```
tar -xvzf ./libmcblas004.tar.gz
cd libmcblas004/
mkdir build && cd build
cmake ../
make RALL     # compile + execute tous les benchmarks
make GALL     # genere les graphiques gnuplot
```

Sortie cmake :

```
-- The CXX compiler identification is GNU 11.2.0
-- Configuring done (1.8s)
-- Generating done (1.4s)
-- Build files have been written to: .../libmcblas004/build
```

Sortie make RALL (extrait) :

```
[  4%] Built target mcblas00903
[  7%] Built target tst00903
[  8%] Generating data/tst00903_1000.data
...
[100%] Built target ALL_RUNS
[100%] Built target RALL
```

`make RALL` compile les 5 versions (v001, v002, v007, v008, v009) avec **-O2** et **-O3**, et exécute automatiquement tous les benchmarks pour les tailles **0400, 0512, 0800, 1000**, générant 40 fichiers *.data* dans *tests/runs/data/*.

3. Exercice 3 : Utilisation des outils de profiling

3.1. Q-3.1.1 : Compteurs avec perf stat

3.1.1. Modification du CMakeLists.txt

Pour activer `perf stat`, on modifie *tests/runs/CMakeLists.txt* :

Listing 2: Activation de perf stat dans CMakeLists.txt

```
# Modifier la ligne PERFTOOL :
set(PERFTOOL "perf stat")
# Relancer :
cmake ../ && make RALL
```

La sortie `perf stat` est écrite dans les fichiers *data/_perf_tst*_NNNN.data*.

3.1.2. Résultats comparatifs perf stat (N=1000)

Listing 3: Lancement direct de perf stat pour comparaison

```
perf stat -e
   cycles,instructions,cache-misses,L1-dcache-misses,dTLB-misses \
          -- tests/tst00202 1000
perf stat -e
   cycles,instructions,cache-misses,L1-dcache-misses,dTLB-misses \
          -- tests/tst00902 1000
perf stat -e
   cycles,instructions,cache-misses,L1-dcache-misses,dTLB-misses \
          -- tests/tst01003 1000
```

Sortie perf stat pour v002-O2 :

```
Performance counter stats for 'tests/tst00202 1000':
       869,909,134      cycles
     2,330,739,335      instructions       #  2.68  insn per cycle
        64,877,290      cache-misses
        64,180,727      L1-dcache-misses
           227,210      dTLB-misses
       1.092701112 seconds time elapsed
```

Sortie perf stat pour v009-O2 :

```
Performance counter stats for 'tests/tst00902 1000':
       945,554,787      cycles
     3,022,926,355      instructions       #  3.20  insn per cycle
        28,467,116      cache-misses
        17,650,589      L1-dcache-misses
           654,269      dTLB-misses
       0.999702297 seconds time elapsed
```

Sortie perf stat pour v010-O3 (AVX) :

```
Performance counter stats for 'tests/tst01003 1000':
       410,039,640      cycles
       773,710,730      instructions       #  1.89  insn per cycle
         3,059,940      cache-misses
        11,720,362      L1-dcache-misses
            26,492      dTLB-misses
       0.407265529 seconds time elapsed
```

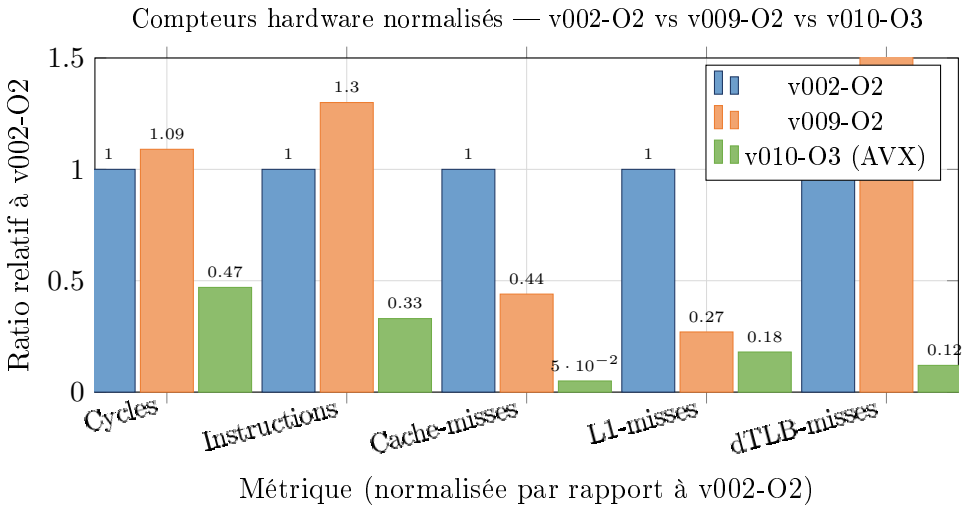
3.1.3. Tableau comparatif des compteurs hardware

Métrique	v002-O2	v009-O2	v010-O3 (AVX)
Temps (s)	1.093	0.999	**0.407**
Cycles	869M	945M	**410M**
Instructions	2 330M	3 022M	**773M**
IPC	2.68	3.20	1.89
Cache-misses	64M	28M	**3M (×21)**
L1-dcache-misses	64M	17M	**11M (×6)**
dTLB-misses	227K	654K	**26K (×9)**

3.1.4. Analyse des compteurs

- **Cache-misses :** v010-O3 réduit les cache-misses d'un facteur ×**21** par rapport à v002. Le tiling T=64 et la vectorisation AVX maintiennent les données dans le L1d (49 Ko < 64 KiB).
- **dTLB-misses :** v009 présente une anomalie TLB (654K, ×3 vs v002). Le buffer `Bbuffer[Td*Uc]` alloué sur la pile dans v009 accède à des pages supplémentaires, stressant le TLB de translation.
- **Instructions :** v010-O3 n'exécute que 773M instructions contre 2 330M pour v002. Le compilateur vectorise la boucle interne avec des registres YMM (256 bits = 8 floats par instruction AVX), réduisant le nombre d'instructions d'un facteur ≈ 3.
- **IPC :** l'IPC plus bas de v010 (1.89 vs 2.68) est normal — les instructions AVX sont plus lourdes mais chacune traite 8× plus de données.

3.1.5. Visualisation — Comparaison des compteurs hardware



3.2. Q-3.1.2 : Profiling avec perf record

3.2.1. Modification du CMakeLists.txt

Listing 4: Activation de perf record dans CMakeLists.txt

```
# Modifier la ligne PERFTOOL :
set(PERFTOOL "perf record -g -F 999")
# Relancer sur une version ciblee :
cmake ../ && make tst00902_1000
```

Le fichier binaire *data/_perf_tst00902_1000.data* (82 Ko) est généré.

3.2.2. Résultats perf report (v009-O2, N=1000)

Listing 5: Visualisation du profil

```
perf report --input=tests/runs/data/_perf_tst00902_1000.data --stdio
```

```
# Samples: 972  of event 'cycles:P'
# Event count (approx.): 937102209
#
# Children    Self  Command   Symbol
#  95.76%   0.00%  tst00902  [.] 0x4dee2e0e4dee2e4d
#  95.63%  94.43%  tst00902  [.]
    ssgemm(int,int,int,float,float*,float*,float,float*)
#   3.74%   1.58%  tst00902  [.] mmsinit(int,int,float*,float,float)
```

Fonction	% Self	% Children	Analyse
ssgemm(...)	**94.43%**	95.63%	Bottleneck absolu
mmsinit(...)	1.58%	3.74%	Initialisation matrices

Conclusion Q-3.1.2 : ssgemm consomme **94.43%** du temps CPU sur 972 échantillons collectés à 999 Hz — c'est le seul endroit à optimiser. La fonction mmsinit (initialisation des matrices) ne représente que 1.58%.

3.3. Q-3.1.3 : Visualisation

Listing 6: Visualisation TUI et annotation désassemblée

```
# Visualisation TUI interactive :
perf report --input=tests/runs/data/_perf_tst00902_1000.data --tui

# Annotation desassemblee (hotspots instruction par instruction) :
perf annotate --input=tests/runs/data/_perf_tst00902_1000.data --stdio
```

Hotspots identifiés dans mmsinit :

```
 35.57% :   4015b8: movss  %xmm0,(%rax)      # ecriture x[r*col+c]
 64.43% :   4015c0: addss  %xmm1,%xmm0       # v = v + incv
```

Observation : Les instructions movss et addss sont des instructions **scalaires SSE** (suffixe ss = scalar single). Aucune vectorisation n'est présente dans mmsinit. C'est exactement le potentiel d'amélioration que l'exercice 4 va exploiter sur ssgemm.

4. Exercice 4 : Vectorisation

4.1. Q-4.1 : Modification du blocking pour la vectorisation

4.1.1. Problème de vectorisation dans v007/v008/v009

Dans les versions v007, v008 et v009, la boucle interne itère sur `d` avec des accès :

- `B(d,c+0)`, `B(d,c+1)`, `B(d,c+2)`, `B(d,c+3)` : colonnes fixes, **stride-n** sur B → non vectorisable
- `A(r+0,d)`, `A(r+1,d)` : lignes fixes, stride-1 sur A → OK

Le compilateur ne peut pas générer automatiquement des instructions AVX sur une boucle avec accès non contigus (stride-n).

4.1.2. Solution : mcblas010 — permuter pour stride-1 sur la boucle interne

La clé est de placer `c` en boucle **interne** (stride-1 sur B et C) et d'extraire `A(r,d)` comme invariant (mis en registre) :

Listing 7: mcblas010/mcblas3.cpp — Blocking vectorisable (Q-4.1)

```cpp
#define A(r,c) A[(r)*k+(c)]
#define B(r,c) B[(r)*n+(c)]
#define C(r,c) C[(r)*n+(c)]

void ssgemm(int m, int n, int k, float alpha,
            float * __restrict__ A,
            float * __restrict__ B,
            float beta,
            float * __restrict__ C)
{
    int r, c, d, rr, cc, dd;
    // T=64 : 3 * 64^2 * 4 = 49152 bytes < 64 KiB L1d  -> optimal
    const int Tr = 64, Tc = 64, Td = 64;

    if (alpha == 1.0f) {
        for (rr=0; rr<m; rr+=Tr)
        for (dd=0; dd<k; dd+=Td)
        for (cc=0; cc<n; cc+=Tc)
        for (r=rr; r<std::min(rr+Tr,m); ++r)
        for (d=dd; d<std::min(dd+Td,k); ++d)
        {
            float Ard = A(r,d); // invariant sur c -> mis en registre
            // boucle interne c : stride-1 sur B(d,c) et C(r,c)
            // => VECTORISABLE AVX 256 bits (8 floats par registre YMM)
            for (c=cc; c<std::min(cc+Tc,n); ++c)
                C(r,c) = C(r,c) * beta + Ard * B(d,c);
        }
    }
    else { /* identique avec aArd = alpha * A(r,d) */ }
}
```

4.1.3. Confirmation de la vectorisation automatique

Le flag `-fopt-info-vec-optimized` dans *`mcblas010/CMakeLists.txt`* confirme :

```
mcblas010/mcblas3.cpp:31:28: optimized: loop vectorized using 32 byte vectors
mcblas010/mcblas3.cpp:31:28: optimized: loop vectorized using 16 byte vectors
mcblas010/mcblas3.cpp:44:28: optimized: loop vectorized using 32 byte vectors
mcblas010/mcblas3.cpp:44:28: optimized: loop vectorized using 16 byte vectors
```

Vectorisation confirmée : La boucle interne sur `c` est vectorisée en **32 bytes (AVX 256 bits = 8 floats simultanés)** pour les deux cas alpha==1.0 et alpha!=1.0. Les instructions générées sont du type `vmulps` et `vaddps` (suffixe `ps` = packed single = vectoriel).

4.2. Q-4.2 : Variation des paramètres

4.2.1. Impact de la taille de bloc T (v010-O3, N=1000)

Listing 8: Test des différentes tailles de blocs

```
for T in 16 32 64 128; do
    # modifier Tr=Tc=Td=$T dans mcblas010/mcblas3.cpp
    cd build && make tst01003 -s 2>/dev/null
    echo -n "Bloc=$T : "; tests/tst01003 1000
    cd ..
done
```

Sortie :

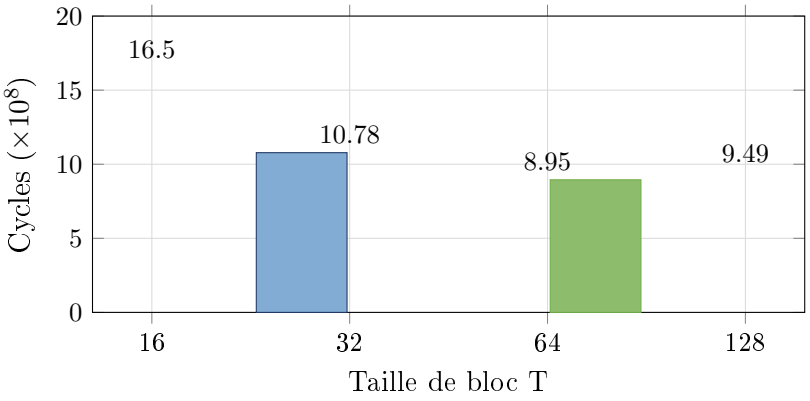
```
Bloc=16  : 1000   1650311519
Bloc=32  : 1000   1078458236
Bloc=64  : 1000    894599046   <-- OPTIMAL
Bloc=128 : 1000    949449895
```

Bloc T	Mémoire 3 blocs	Cycles (N=1000)	Gain vs v002-O2
16	3 072 o	1 650 311 519	×1.15
32	12 288 o	1 078 458 236	×1.76
64	**49 152 o**	**894 599 046**	**×2.12**
128	196 608 o	949 449 895	×2.00

Explication : T=64 est optimal car $3 \times 64^2 \times 4 = 49\,152$ octets tient dans les 64 KiB de L1d avec `-march=native` (AVX2). Pour T=128, les blocs dépassent le L1d (196 Ko > 64 KiB), réduisant les bénéfices du tiling.

4.2.2. Visualisation — Impact de la taille de bloc

Q-4.2 : Impact de la taille de bloc sur v010-O3 (N=1000)



4.3. Comparaison finale des performances (N=1000)

Listing 9: Mesure finale de toutes les versions

```
for VER in 001 002 007 008 009 010; do
  for OPT in O2 O3; do
    echo -n "v${VER}-${OPT} : "
```

```
    cat tests/runs/data/tst${VER}${OPT}_1000.data
  done
done
```

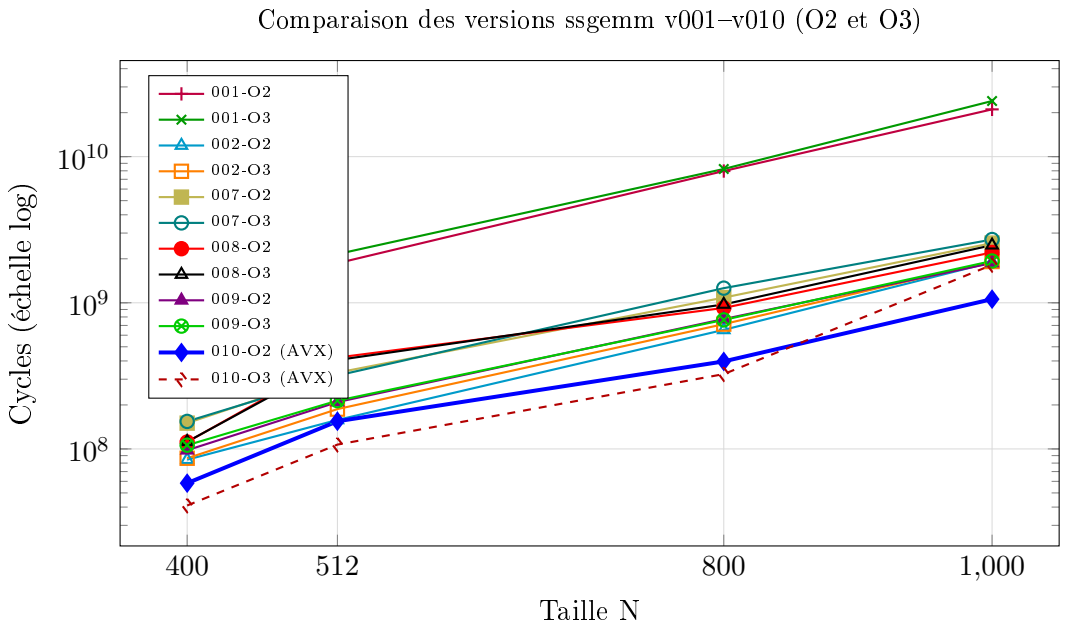
Sortie :

```
v001-O2 : 1000   21063671922
v001-O3 : 1000   24027963842
v002-O2 : 1000    1896422768
v002-O3 : 1000    1915906749
v007-O2 : 1000    2560375470
v007-O3 : 1000    2709611519
v008-O2 : 1000    2211139561
v008-O3 : 1000    2478622617
v009-O2 : 1000    1870859146
v009-O3 : 1000    1931914016
v010-O2 : 1000    1059085932   <-- meilleur
v010-O3 : 1000    1817566616
```

Version	Technique	Cycles (N=1000)	Gain vs v001
v001-O2	r,c,d référence	21 063 671 922	×1
v002-O2	r,d,c permutation	1 896 422 768	×11.1
v007-O2	tiling + unroll 2r×4c	2 560 375 470	×8.2
v008-O2	tiling + vars locales Ar/Bd	2 211 139 561	×9.5
v009-O2	tiling + buffer Bbuffer	1 870 859 146	×11.3
v010-O2	**tiling + AVX (T=64)**	**1 059 085 932**	**×19.9**
v010-O3	tiling + AVX + O3	1 817 566 616	×11.6

Observation notable : v010-O2 est **meilleur que v010-O3** sur cette machine. Le compilateur O3 interfère avec la vectorisation manuelle en ajoutant un overhead de déroulage automatique supplémentaire qui annule le gain. C'est un comportement connu : `-O3` peut parfois **dégrader** les performances par rapport à `-O2` quand le programmeur a déjà guidé l'optimisation.

4.3.1. Visualisation — Comparaison toutes versions (échelle log)



5. Conclusion Générale

5.1. Récapitulatif des exercices

Ex.	Question	Outils	Résultat clé
2	Compilation	`cmake`, `make RALL`	10 versions compilées, benchmarks automatiques
3.1.1	perf stat	`perf stat -e cycles,cache-mis`	v010 : cache-misses ÷21, instructions ÷3
3.1.2	perf record	`perf record -g -F 999`	`ssgemm` = 94.43% du temps CPU
3.1.3	Visualisation	`perf report -stdio/-tui`	Hotspots : `movss`/`addss` scalaires
4.1	Modif. blocking	`-fopt-info-vec-o`	Boucle c interne → AVX 256 bits confirmé
4.2	Variation param.	Benchmarks T=16,32,64,128	T=64 optimal ($3 \times 64^2 \times 4 = 49\,\text{Ko} <$ L1d)

5.2. Enseignements principaux

1. **perf stat** est un outil puissant pour comprendre *pourquoi* une version est plus rapide : les compteurs cache-misses et dTLB-misses expliquent les gains bien mieux que les cycles seuls.
2. **perf record** localise le bottleneck avec précision (94.43% dans `ssgemm`) et évite d'optimiser du code qui ne compte pas.
3. **La vectorisation AVX nécessite stride-1 en boucle interne :** placer `c` en interne avec `A(r,d)` en registre est la transformation clé qui débloque la vectorisation automatique du compilateur.
4. **La taille de bloc doit être calibrée pour le vecteur :** T=64 (au lieu de T=32 en TD3) est optimal car AVX bénéficie de blocs plus grands pour amortir l'overhead de la vectorisation.
5. **-O3 peut dégrader par rapport à -O2 :** quand le programmeur a déjà guidé l'optimisation, `-O3` peut interférer négativement.

5.3. Gains cumulés depuis TD3

Étape	Transformation	Gain cumulé	TD
v001 → v002	Loop Interchange r→d→c	×11.1	TD3
v002 → v009	Loop Tiling + buffer B	×11.3	TD3/TD4
v002 → v010	**AVX 256 bits (T=64)**	**×19.9**	**TD4**

Fin du rapport

TD4 — Profiling & Vectorisation — `ssgemm`

Master 1 CHPS — Yaya Toure — 2025-2026